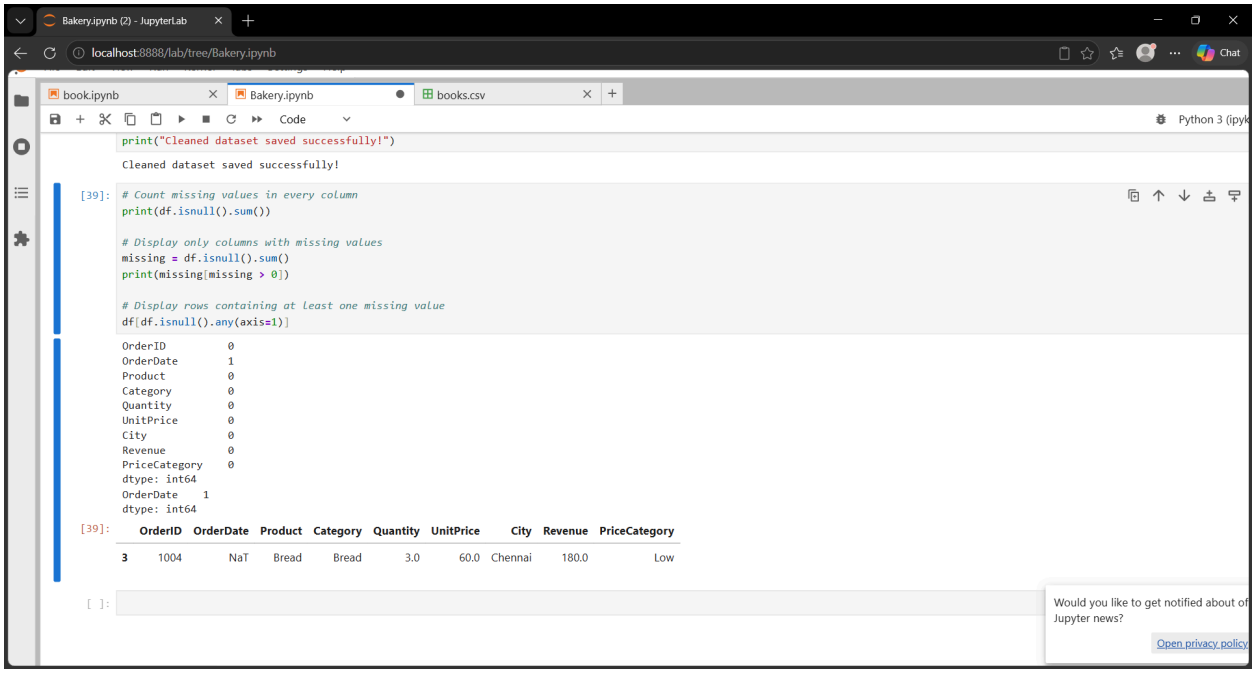
1. Identify missing values - isnull().sum()

Use: Identifying missing values helps us understand whether the dataset is complete and reliable. In bakery sales data, columns such as Quantity, UnitPrice, OrderDate, and City are important for analysing sales. If values are missing, calculations and reports may become inaccurate. The isnull().sum() command counts the missing values in every column, helping us identify exactly where the problem exists.

Possible causes:

- The employee forgot to enter the quantity or price when recording an order.
- A customer placed an order, but some details were not provided.
- Data was imported incorrectly from an Excel file or another system.
- Invalid values, such as not_a_date, were converted into missing dates using errors="coerce".
- A product's price was recorded as text, such as "unknown", and later converted into a numeric column.

Example: If the Quantity column has two missing values, we cannot calculate the exact revenue for those orders until we find or estimate the missing quantities.




```
[40]:
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["UnitPrice"] = df["UnitPrice"].fillna(df["UnitPrice"].median())

[41]:
df["UnitPrice"] = pd.to_numeric(df["UnitPrice"], errors="coerce")
```

2. Identify duplicate orders - duplicated()

Use: Identifying duplicates helps us find repeated records in a dataset. In bakery sales data, the same order may accidentally be entered more than once. Duplicate records can increase the calculated number of orders and inflate total revenue, making the business appear to have more sales than it actually did.
Possible causes:

- An employee entered the same order twice.
- The same file was imported into the system more than once.
- Data was copied and pasted incorrectly.
- An automated sales system sent the same record multiple times.
- Multiple items legitimately belong to the same order ID, so a repeated ID does not always mean a duplicate.

Example: If OrderID 1005 appears twice with identical product, quantity, price, and city details, it may be a duplicate record. We should inspect the rows before removing them because two different items can sometimes share the same order ID.



3. Find the highest unit price - max()

Use: Finding the maximum unit price helps us identify the most expensive product in the dataset. A bakery can use this information to understand its premium products, review its pricing, and compare product prices. The max() method returns the highest available numeric value in a column.
Possible causes of a very high price:

- The product is genuinely expensive because it uses premium ingredients.
- The product is a large or customised cake.
- An employee accidentally added an extra zero while entering the price.
- A currency or decimal-formatting mistake occurred during data entry.
- Text values such as "unknown" have not been converted into proper numeric values.

Example: If the highest unit price is ₹45,000 when most bakery products cost between ₹40 and ₹450, it could be a data-entry mistake. However, we should verify the original order before changing it.
Important distinction: The highest unit price identifies the most expensive individual product, whereas the highest revenue identifies the order that generated the most sales value.

[42]:

```
print("Highest unit price:", df["UnitPrice"].max())
```

```
Highest unit price: 450.0
```

[43]:

```
df["Revenue"] = df["Quantity"] * df["UnitPrice"]

highest_revenue = df["Revenue"].max()

print(highest_revenue)
```

```
900.0
```

4. Group revenue by city - groupby()

Use: Grouping revenue by city helps a bakery understand which locations generate the most sales. This information supports business decisions such as allocating stock, planning deliveries, running promotions, and deciding where to expand operations. By grouping the data by City and adding the Revenue values, we can compare the total revenue generated in each location.
Possible causes of differences in revenue between cities:

- One city has more customers than another.
- Some locations receive larger orders or sell more products.
- Certain products are more popular in particular cities.
- Prices, discounts, or seasonal demand vary by location.
- Some orders have missing quantity or price values, making their revenue incomplete.
- Inconsistent city names, such as Chennai, chennai, and Chennai, split the same location into separate groups if the text is not standardised.
- Duplicate orders inflate the revenue for a city.

Example: If Chennai generates ₹5,000 in revenue and Bangalore generates ₹3,000, Chennai has higher recorded revenue for the dataset. The bakery can investigate whether this is due to more orders, larger quantities, higher-priced products, or greater customer demand.

[45]:

```
city_revenue = (
    df.groupby("City")["Revenue"]
    .sum()
    .sort_values(ascending=False)
)

print(city_revenue)
```

```
City
Chennai       2400.0
Madurai        880.0
Bangalore      480.0
Coimbatore     300.0
Name: Revenue, dtype: float64
```